Java Web Development Assignment

Project: Smart Expense Tracker

Difficulty: Medium
Technologies: HTML, CSS, Java Servlet, JDBC, MySQL, Apache Tomcat
Restriction: No JSP, no DAO, no Spring, and no ORM.

1. Objective

Create a web application that allows users to record daily expenses, search previous expenses, view their spending, and delete incorrect records.

Use Servlets to handle requests and JDBC directly inside the Servlet classes to communicate with MySQL.

2. Main Menu

The home page should contain only four options:

1. Add Expense
2. View Expenses
3. Search Expense
4. Spending Summary

3. Functional Requirements

A. Add Expense

Create an HTML form with the following fields:

- Expense Title
- Amount
- Category: Food, Travel, Education, Shopping, Other
- Expense Date
- Payment Method: Cash, UPI, Card

Requirements:

- Validate all fields.
- Amount must be greater than zero.
- Save the expense in MySQL using JDBC.
- Display a success or error message.

B. View Expenses

Display all expenses in a dynamically generated HTML table.

Columns: ID, Title, Amount, Category, Date, Payment Method, Action.

Requirements:

- Retrieve records using JDBC and `ResultSet`.
- Display the newest expenses first.
- Provide a Delete button for each record.
- Display a suitable message if no records exist.

C. Search Expense

Allow users to search by expense title or category.

Example: Searching for `Food` should display all expenses belonging to that category.

Requirements:

- Use `PreparedStatement`.
- Display matching records in an HTML table.
- Display `No Expenses Found` when no records match.

D. Spending Summary

Display the following information calculated from the database:

- Total money spent.
- Total number of expenses.
- Highest individual expense.
- Category with the highest total spending.

Example:

```
SMART SPENDING SUMMARY

Total Spent              : ₹7,500
Number of Expenses       : 15
Highest Individual Cost  : ₹2,000
Top Spending Category    : Shopping
```

These values must come from SQL queries, not hardcoded values.

4. Database Setup

Run the following SQL script in MySQL:

```
CREATE DATABASE expense_tracker;
USE expense_tracker;

CREATE TABLE expenses (
    id INT PRIMARY KEY AUTO_INCREMENT,
    title VARCHAR(100) NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    category VARCHAR(50) NOT NULL,
    expense_date DATE NOT NULL,
```

```
    payment_method VARCHAR(20) NOT NULL
);
```

5. Technical Rules

Students must follow these rules:

1. Use HTML forms to submit data to Servlets.
2. Use `HttpServlet` and implement the appropriate `doGet()` and `doPost()` methods.
3. Use JDBC directly inside the Servlet classes.
4. Use `DriverManager`, `Connection`, `PreparedStatement`, and `ResultSet`.
5. Use `executeUpdate()` for INSERT and DELETE operations.
6. Use `executeQuery()` for SELECT and summary queries.
7. Use `PreparedStatement` for all SQL queries involving user input.
8. Use try-with-resources to close JDBC resources.
9. Validate data on the server side.
10. Generate HTML dynamically from Servlets using `response.getWriter()`.
11. Use POST for operations that modify database records.
12. Redirect after a successful insert or delete to prevent accidental duplicate submissions.

Important: No JSP, DAO class, or separate database utility class is allowed. Database connection and SQL operations must be handled directly in the relevant Servlet classes.

6. Expected Project Structure

```
ExpenseTracker/
│
├── src/main/java/
│   └── com/expense/
│       ├── AddExpenseServlet.java
│       ├── ViewExpenseServlet.java
│       ├── SearchExpenseServlet.java
│       ├── DeleteExpenseServlet.java
│       └── SummaryServlet.java
│
└── src/main/webapp/
    ├── index.html
    ├── addExpense.html
    └── searchExpense.html
```

Students may create additional HTML and CSS files if required. The output tables and summary can be generated directly by the Servlets.

7. Evaluation Criteria

Criteria	Marks
Database connection and JDBC	15

Criteria	Marks
Add Expense	15
View and Delete Expenses	20
Search functionality	15
Spending Summary	15
Validation and error handling	10
UI and successful demonstration	10
Total	**100**

8. Bonus Challenge

Implement a **Monthly Budget Alert**. Let the user enter a monthly budget, calculate the actual spending for that month, and display a warning if the spending exceeds the budget.

Final Submission

Submit the complete source code, database SQL script, and screenshots demonstrating all four menu options.

The application must run successfully on Apache Tomcat and use actual MySQL records throughout.